



Database Management System Lab (BTCS203P)

SQL Joins

Below is the complete lab sheet in the **same format as before: Question → Query → Output.**

Assumed Tables

Student

StudentID	Name	DeptID	CourseID
1	Amit	1	101
2	Priya	2	102
3	Rahul	1	103
4	Neha	NULL	101
5	Anjali	3	102

Department

DeptID	DeptName
1	CSE
2	AI&ML
3	Cyber Security
4	Data Science

Course

CourseID	CourseName
101	DBMS
102	Python
103	Data Structures

Employee

EmployeeID	Name	DeptID	ManagerID	Salary
1	Raj	1	NULL	80000
2	Simran	1	1	50000

EmployeeID	Name	DeptID	ManagerID	Salary
3	Karan	2	1	60000
4	Neha	2	3	45000

1. Demonstrate an INNER JOIN between Student and Department

Query:

```
SELECT Student.Name, Department.DeptName
FROM Student
INNER JOIN Department
ON Student.DeptID = Department.DeptID;
```

Output:

Name	DeptName
Amit	CSE
Priya	AI&ML
Rahul	CSE
Anjali	Cyber Security

2. Display student names along with their department names using INNER JOIN

Query:

```
SELECT s.Name, d.DeptName
FROM Student s
INNER JOIN Department d
ON s.DeptID = d.DeptID;
```

Output:

Name	DeptName
Amit	CSE
Priya	AI&ML
Rahul	CSE
Anjali	Cyber Security

3. Display employee names along with their department names

Query:

```
SELECT e.Name, d.DeptName
FROM Employee e
INNER JOIN Department d
ON e.DeptID = d.DeptID;
```

Output:

Name	DeptName
Raj	CSE
Simran	CSE
Karan	AI&ML
Neha	AI&ML

4. Display students along with their course details

Query:

```
SELECT s.Name, c.CourseID, c.CourseName
FROM Student s
INNER JOIN Course c
ON s.CourseID = c.CourseID;
```

Output:

Name	CourseID	CourseName
Amit	101	DBMS
Priya	102	Python
Rahul	103	Data Structures
Neha	101	DBMS
Anjali	102	Python

5. Find students belonging to a particular department using INNER JOIN

Query:

```
SELECT s.Name, d.DeptName
FROM Student s
INNER JOIN Department d
ON s.DeptID = d.DeptID
WHERE d.DeptName = 'CSE';
```

Output:

Name	DeptName
Amit	CSE
Rahul	CSE

6. Demonstrate a LEFT OUTER JOIN between Student and Department

Query:

```
SELECT s.Name, d.DeptName
FROM Student s
LEFT JOIN Department d
ON s.DeptID = d.DeptID;
```

Output:

Name	DeptName
Amit	CSE
Priya	AI&ML
Rahul	CSE
Neha	NULL
Anjali	Cyber Security

7. Display all students, including those who do not have a matching department

Query:

```
SELECT s.Name, d.DeptName
FROM Student s
LEFT JOIN Department d
ON s.DeptID = d.DeptID;
```

Output:

Name	DeptName
Amit	CSE
Priya	AI&ML
Rahul	CSE
Neha	NULL
Anjali	Cyber Security

8. Demonstrate a RIGHT OUTER JOIN between Student and Department

Query:

```
SELECT s.Name, d.DeptName
FROM Student s
RIGHT JOIN Department d
ON s.DeptID = d.DeptID;
```

Output:

Name	DeptName
Amit	CSE
Rahul	CSE
Priya	AI&ML
Anjali	Cyber Security
NULL	Data Science

9. Display all departments, including departments having no students

Query:

```
SELECT d.DeptName, s.Name
FROM Student s
RIGHT JOIN Department d
ON s.DeptID = d.DeptID;
```

Output:

DeptName	Name
----------	------

CSE	Amit	
CSE	Rahul	
AI&ML	Priya	
Cyber Security	Anjali	
Data Science	NULL	
+-----+	+-----+	

10. Find students who are not assigned to any department using LEFT JOIN

Query:

```
SELECT s.Name
FROM Student s
LEFT JOIN Department d
ON s.DeptID = d.DeptID
WHERE d.DeptID IS NULL;
```

Output:

+-----+
Name
+-----+
Neha
+-----+

11. Find departments having no students using RIGHT/LEFT JOIN

Query:

```
SELECT d.DeptName
FROM Department d
LEFT JOIN Student s
ON d.DeptID = s.DeptID
WHERE s.StudentID IS NULL;
```

Output:

+-----+
DeptName
+-----+
Data Science
+-----+

12. Demonstrate a FULL OUTER JOIN concept using suitable queries/workaround

Query:

```
SELECT s.Name, d.DeptName
FROM Student s
LEFT JOIN Department d
ON s.DeptID = d.DeptID
```

UNION

```
SELECT s.Name, d.DeptName
FROM Student s
RIGHT JOIN Department d
ON s.DeptID = d.DeptID;
```

Output:

Name	DeptName
Amit	CSE
Priya	AI&ML
Rahul	CSE
Neha	NULL
Anjali	Cyber Security
NULL	Data Science

13. Demonstrate a CROSS JOIN between Student and Course

Query:

```
SELECT s.Name, c.CourseName
FROM Student s
CROSS JOIN Course c;
```

Output:

Name	CourseName
Amit	DBMS
Amit	Python
Amit	Data Structures
Priya	DBMS
Priya	Python
Priya	Data Structures
Rahul	DBMS

Rahul	Python
Rahul	Data Structures
Neha	DBMS
Neha	Python
Neha	Data Structures
Anjali	DBMS
Anjali	Python
Anjali	Data Structures

14. Calculate the number of possible Student-Course combinations using CROSS JOIN

Query:

```
SELECT COUNT(*) AS TotalCombinations
FROM Student
CROSS JOIN Course;
```

Output:

TotalCombinations
15

15. Demonstrate a NATURAL JOIN between two suitable tables

Query:

```
SELECT *
FROM Student
NATURAL JOIN Department;
```

Output:

DeptID	Name	CourseID	DeptName
1	Amit	101	CSE
2	Priya	102	AI&ML
1	Rahul	103	CSE
3	Anjali	102	Cyber Security

16. Compare the result of NATURAL JOIN and INNER JOIN

NATURAL JOIN:

```
SELECT *
FROM Student
NATURAL JOIN Department;
```

INNER JOIN:

```
SELECT s.StudentID, s.Name, s.DeptID, s.CourseID, d.DeptName
FROM Student s
INNER JOIN Department d
ON s.DeptID = d.DeptID;
```

Output:

Both queries return the matching students and departments.

ID	Name	DeptID	CourseID	DeptName
1	Amit	1	101	CSE
2	Priya	2	102	AI&ML
3	Rahul	1	103	CSE
5	Anjali	3	102	Cyber Security

17. Perform a SELF JOIN on an Employee table to show employee-manager relationships

Query:

```
SELECT e.Name AS Employee,
       m.Name AS Manager
FROM Employee e
LEFT JOIN Employee m
ON e.ManagerID = m.EmployeeID;
```

Output:

Employee	Manager
Raj	NULL
Simran	Raj
Karan	Raj

Neha	Karan	
+-----+	+-----+	+

18. Display employees along with their managers using SELF JOIN

Query:

```
SELECT e.Name AS EmployeeName,  
       m.Name AS ManagerName  
FROM Employee e  
LEFT JOIN Employee m  
ON e.ManagerID = m.EmployeeID;
```

Output:

+-----+	+-----+	+
EmployeeName	ManagerName	
+-----+	+-----+	+
Raj	NULL	
Simran	Raj	
Karan	Raj	
Neha	Karan	
+-----+	+-----+	+

19. Join three tables—Student, Department and Course

Query:

```
SELECT s.Name, d.DeptName, c.CourseName  
FROM Student s  
INNER JOIN Department d  
ON s.DeptID = d.DeptID  
INNER JOIN Course c  
ON s.CourseID = c.CourseID;
```

Output:

+-----+	+-----+	+-----+	+
Name	DeptName	CourseName	
+-----+	+-----+	+-----+	+
Amit	CSE	DBMS	
Priya	AI&ML	Python	
Rahul	CSE	Data Structures	
Anjali	Cyber Security	Python	
+-----+	+-----+	+-----+	+

20. Display student name, course name and department name using multiple joins

Query:

```
SELECT s.Name AS StudentName,  
       c.CourseName,  
       d.DeptName  
FROM Student s  
INNER JOIN Course c  
ON s.CourseID = c.CourseID  
INNER JOIN Department d  
ON s.DeptID = d.DeptID;
```

Output:

StudentName	CourseName	DeptName
Amit	DBMS	CSE
Priya	Python	AI&ML
Rahul	Data Structures	CSE
Anjali	Python	Cyber Security

21. Find employees whose salary is greater than the average salary of their department

Query:

```
SELECT e.Name, e.Salary, d.DeptName  
FROM Employee e  
INNER JOIN Department d  
ON e.DeptID = d.DeptID  
WHERE e.Salary > (  
    SELECT AVG(e2.Salary)  
    FROM Employee e2  
    WHERE e2.DeptID = e.DeptID  
);
```

Output:

Name	Salary	DeptName
Raj	80000	CSE
Karan	60000	AI&ML

22. Find the department having the maximum number of students using joins and aggregation

Query:

```
SELECT d.DeptName, COUNT(s.StudentID) AS TotalStudents
FROM Department d
LEFT JOIN Student s
ON d.DeptID = s.DeptID
GROUP BY d.DeptID, d.DeptName
ORDER BY TotalStudents DESC
LIMIT 1;
```

Output:

DeptName	TotalStudents
CSE	2

23. Display departments and the total number of students in each department

Query:

```
SELECT d.DeptName,
       COUNT(s.StudentID) AS TotalStudents
FROM Department d
LEFT JOIN Student s
ON d.DeptID = s.DeptID
GROUP BY d.DeptID, d.DeptName;
```

Output:

DeptName	TotalStudents
CSE	2
AI&ML	1
Cyber Security	1
Data Science	0

24. Display courses along with the number of students enrolled in each course

Query:

```
SELECT c.CourseName,
       COUNT(s.StudentID) AS TotalStudents
FROM Course c
LEFT JOIN Student s
ON c.CourseID = s.CourseID
GROUP BY c.CourseID, c.CourseName;
```

Output:

CourseName	TotalStudents
DBMS	2
Python	2
Data Structures	1

25. Create a complete query using INNER JOIN, LEFT JOIN, GROUP BY and aggregate functions to generate a student-department-course report

Query:

```
SELECT d.DeptName,
       c.CourseName,
       COUNT(s.StudentID) AS TotalStudents
FROM Department d
LEFT JOIN Student s
ON d.DeptID = s.DeptID
INNER JOIN Course c
ON s.CourseID = c.CourseID
GROUP BY d.DeptName, c.CourseName;
```

Output:

DeptName	CourseName	TotalStudents
CSE	DBMS	1
CSE	Data Structures	1
AI&ML	Python	1
Cyber Security	Python	1

+-----+-----+-----+